Therefore running `listmachine ex30.out 1000` will fail with the message `Out of memory` because everything the program (`ex30.out` from Sect. 10.7.1) allocates in the heap will remain there forever.

Exercise 10.1 To understand how the abstract machine and the garbage collector work and how they collaborate, answer these questions:

(i) Write 3–10 line descriptions of how the abstract machine executes each of the following instructions:

- `ADD`, which adds two integers.
- `CSTI i`, which pushes integer constant `i`.
- `NIL`, which pushes a `nil` reference. What is the difference between `NIL` and `CSTI 0`?
- `IFZERO`, which tests whether an integer is zero, or a reference is `nil`.
- `CONS`
- `CAR`
- `SETCAR`

(ii) Describe the result of applying each C macro `Length`, `Color` and `Paint` from Sect. 10.7.4 to a block header `ttttttttnnnnnnnnnnnnnnnnnnnnngg`, that is, a 32-bit word, as described in the source code comments.
(iii) When does the abstract machine, or more precisely, its instruction interpretation loop, call the `allocate(...)` function? Is there any other interaction between the abstract machine (also called the mutator) and the garbage collector?
(iv) In what situation will the garbage collector's `collect(...)` function be called?

Exercise 10.2 Add a simple mark-sweep garbage collector to listmachine.c, like this:

```
void collect(int s[], int sp) {
  markPhase(s, sp);
  sweepPhase();
}
```

Your `markPhase` function should scan the abstract machine stack `s[0..sp]` and call an auxiliary function `mark(word* block)` on each non-nil heap reference in the stack, to mark live blocks in the heap. Function `mark(word* block)` should recursively mark everything reachable from the block.

The `sweepPhase` function should scan the entire heap, put white blocks on the freelist, and paint black blocks white. It should ignore blue blocks; they are either already on the freelist or they are orphan blocks which are neither used for data nor on the freelist, because they consist only of a block header, so there is no way to link them into the freelist.

This may sound complicated, but the complete solution takes less than 30 lines of C code.

Running `listmachine ex30.out 1000` should now work, also for arguments that are much larger than 1000.

Remember that the listmachine has a tracing mode `listmachine -trace ex30.out 4` so you can see the stack state just before your garbage collector crashes.

Also, calling the `heapStatistics()` function in `listmachine.c` performs some checking of the heap's consistency and reports some statistics on the number of used and free blocks and so on. It may be informative to call it before and after garbage collection, and between the mark and sweep phases.

When your garbage collector works, use it to run the list-C programs `ex35.lc` and `ex36.lc` and check that they produce the expected output (described in their source files). These programs build shared and cyclic data structures in the heap, and this may reveal flaws in your garbage collector.

Exercise 10.3 Improve the sweep phase so that it joins adjacent dead blocks into a single dead block. More precisely, when sweep finds a white (dead) block of length n at address p, it checks whether there is also a white block at address $p + 1 + n$, and if so join them into one block.

Don't forget to run the list-C programs `ex35.lc` and `ex36.lc` as in Exercise 10.2.

Exercise 10.4 Further improve the sweep phase so that it can join any number of adjacent dead blocks in to a single dead block. This is important to avoid fragmentation when allocated blocks may be of different sizes.

Exercise 10.5 Change the mark phase function so that it does not use recursion. Namely, the `mark` function may overflow the C stack when it attempts to mark a deep data structure in the heap, such as a long list created from cons cells.

Instead the mark phase must (A) paint grey all blocks that are directly reachable from the stack. Then (B) it should traverse the heap and whenever it finds a grey block `b`, paint it black, and then paint grey all white blocks that are reachable from `b`. The heap traversal must be repeated until there are no more grey blocks in the heap.

So color grey means "this block is live, but the blocks it directly refers to may not have been painted yet", and color black means "this block is live, and all the blocks it directly refers to have been painted grey (or even black)".

Don't forget to run the list-C programs `ex35.lc` and `ex36.lc` as in Exercise 10.2.

Exercise 10.6 Replace the freelist and the mark-sweep garbage collector with a two-space stop-and-copy garbage collector.

The `initheap(…)` function must allocate two heap-spaces (that is, twice as much memory as before), and there must be must be two heap pointers, `heapFrom` and `heapTo`, corresponding to the two heap spaces, and two after-heap pointers `afterFrom` and `afterTo`.

That is, the `freelist` pointer no longer points to a list of unused blocks, but to the first unused word in from-space. All words from that one until (but not including) `afterFrom` are unused. The `allocate(…)` function can therefore be much